# WEKA CSI for Multi-Tenant Kubernetes

Giving a tenant limited, scoped credentials instead of cluster-admin

**Prepared for the Coupang platform team — WEKA Premium Support** Follow-up to the 2026-06-30 status call. All steps below were validated end-to-end against **WEKA 4.4.10** with **CSI plugin v2.8.8** using directory-backed ( `dir/v1` ) volumes.

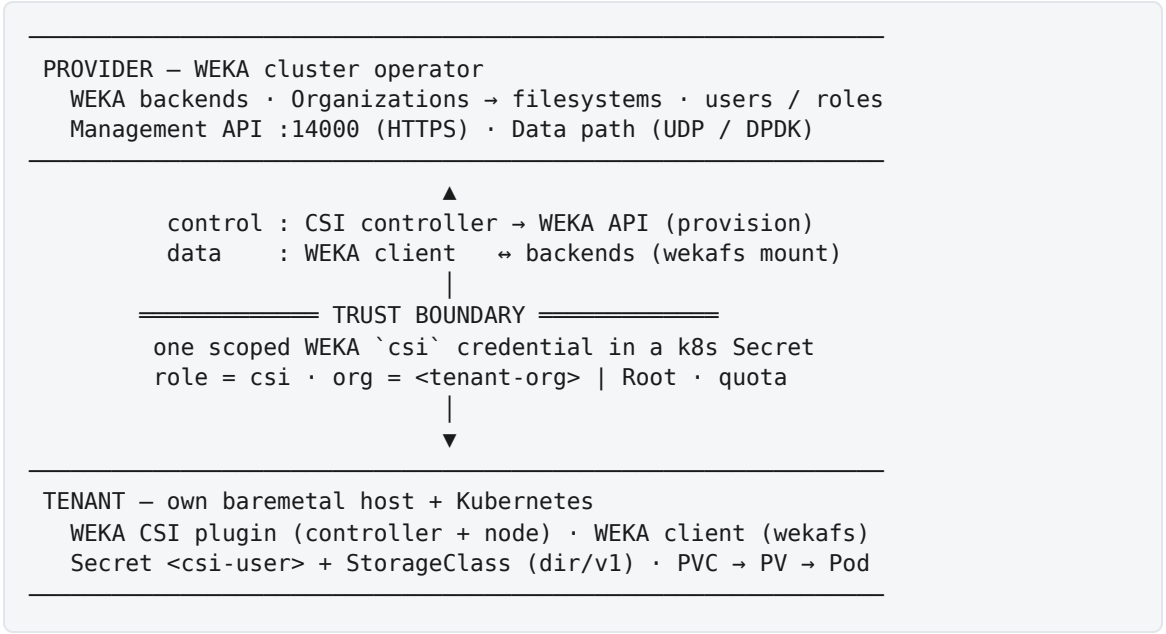
> **Companion repository (full manifests, Terraform, and the complete command transcript):** `github.com/weka/csi-tenant-onboarding-lab`

---

## 1. Goal

Let a tenant run its own Kubernetes cluster and consume storage from a shared WEKA cluster through the WEKA CSI plugin — **without ever handing the tenant cluster-admin credentials.** The tenant should only be able to act on its own slice of the cluster.

## 2. How it fits together



The **trust boundary is a single WEKA API credential** in a Kubernetes Secret in the tenant's cluster. Everything the tenant's CSI can do on WEKA is bounded by that user's **role + organization + capacity quota**.

## 3. The credential: use the dedicated `csi` role

WEKA provides a purpose-built role for exactly this — it grants only the operations the provisioner needs (create/delete directories, quotas, snapshots) and nothing else. Never use `admin`/ClusterAdmin in a tenant Secret.

```
weka user add <username> csi <password>
```

The Kubernetes Secret the tenant receives:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: csi-wekafs-api-secret
  namespace: csi-wekafs
type: Opaque
stringData:
  username: <tenant-csi-user>
  password: <password>
  organization: <tenant-org-or-Root>   # tenant org (Model A) or Root (Model B)
  scheme: https                        # HTTPS is mandatory on WEKA 4.3+
  endpoints: "<backend-ip>:14000,<backend-ip>:14000"
  caCertificate: ""                    # base64 PEM for a private/self-signed
        cert
```

The StorageClass wires this Secret into all five external-provisioner slots and pins the tenant's filesystem:

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: sc-tenant-dir
provisioner: csi.weka.io
reclaimPolicy: Delete
volumeBindingMode: Immediate
allowVolumeExpansion: true
parameters:
  volumeType: dir/v1
  filesystemName: <tenant-fs>
  capacityEnforcement: HARD
  csi.storage.k8s.io/provisioner-secret-name: csi-wekafs-api-secret
  csi.storage.k8s.io/provisioner-secret-namespace: csi-wekafs
  # (controller-publish / controller-expand / node-stage / node-publish: same
        secret)
```

## 4. Two isolation models

| | Model A — dedicated Organization (recommended) | Model B — root org + dedicated filesystem |
|---|---|---|
| Secret `organization` | the tenant's org | `Root` |
| Isolation | hard org boundary — the credential cannot see other orgs' filesystems | fenced only by the StorageClass pin + quota |
| Blast radius | confined to the org + its quota | root-org visibility; relies on role + pin |
| Best for | real multi-tenant platforms | single-tenant / trusted setups |

Both use the `csi` role and directory-backed ( `dir/v1` ) volumes.

## 5. Provider steps (WEKA cluster operator)

Run as ClusterAdmin. Directory-backed provisioning means the operator pre-creates the filesystem; the tenant's `csi` user only creates directories + quotas inside it.

**Model A — dedicated Organization:**

```
# org + org-admin, with a capacity quota (hard multi-tenancy boundary)
weka org add <tenant-org> <tenant-admin> <admin-pw> --ssd-quota 300GB --total-
        quota 300GB

# create the tenant's filesystem inside the org (fs-groups are cluster-level →
        reuse 'default')
weka user login <tenant-admin> <admin-pw> --org <tenant-org>
weka fs add <tenant-fs> default 100GB

# the scoped CSI credential
weka user add <tenant-csi-user> csi <csi-pw>
```

**Model B — root org:**

```
weka fs add <tenant-fs> default 100GB
weka user add <tenant-csi-user> csi <csi-pw>
```

Then hand the tenant: `username` , `password` , `organization` , the backend `endpoints` , `scheme: https` , and a CA cert if the API uses a private cert.

## 6. Tenant steps (Kubernetes side)

**a. Install the WEKA client on each node.** The CSI wekafs transport needs a running WEKA client on the host. Build tools are required (on Ubuntu 22.04 the driver needs **gcc-12**):

```
sudo apt-get install -y gcc-12 make "linux-headers-$(uname -r)"
sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-12 100 && sudo
        update-alternatives --set gcc /usr/bin/gcc-12
curl -s http://<backend-ip>:14000/dist/v1/install | sudo sh
sudo weka version get <cluster-version>
sudo weka local setup container --name client --client \
  --net udp --core-ids <core> --join-ips <backend-ips>
weka local ps          # STATE=Running, STATUS=Ready
```

**b. Install the CSI plugin.**

```
helm repo add csi-wekafs https://weka.github.io/csi-wekafs && helm repo update
helm upgrade --install csi-wekafs csi-wekafs/csi-wekafsplugin \
  --namespace csi-wekafs --create-namespace
  # if the WEKA API uses a self-signed cert, add:
  #   --set pluginConfig.allowInsecureHttps=true   (or provide caCertificate in
        the Secret)
```

**c. Apply the Secret, StorageClass, and a workload**, then verify:

```
kubectl apply -f csi-secret.yaml -f storageclass.yaml -f pvc-and-pod.yaml
kubectl get pvc          # STATUS=Bound
kubectl exec <pod> -- sh -c 'mount | grep /data; cat /data/temp.txt'
```

## 7. What success looks like (validated)

In the lab, a tenant's single-node Kubernetes cluster provisioned and mounted `dir/v1` volumes using only its scoped `csi` credentials:

```
$ kubectl get pvc
NAME           STATUS   VOLUME                     CAPACITY   ACCESS MODES
STORAGECLASS
pvc-tenant-a   Bound    pvc-9cf3dd3a-...           1Gi        RWX
sc-tenant-a-dir

# PV handle — a directory inside the tenant's filesystem:
dir/v1/tenant-a-fs/csi-volumes/pvc-9cf3dd3a-...

$ kubectl exec app-tenant-a -- sh -c 'mount | grep /data'
tenant-a-fs on /data type wekafs (rw,... ,container_name=client)
```

**Organization isolation, proven:** with Model A, a cluster admin listing the root org does *not* see the tenant's filesystem — it exists only inside the tenant's organization.

## 8. Recommendation for Coupang

- Use the dedicated `csi` **role** for all tenant CSI credentials — never admin.
- For multi-tenant isolation, use **Model A (dedicated Organization)**: it adds a hard WEKA-native boundary on top of the least-privilege role.
- Keep provisioning **directory-backed (`dir/v1`)** — it needs the smallest privilege (the operator owns the filesystem; the tenant only makes directories).
- In production, provide the API `caCertificate` rather than `allowInsecureHttps`.

## Appendix — gotchas we hit and resolved

| Symptom | Cause / fix |
|---|---|
| CSI driver won't build | Weka 4.4.10 needs kernel < 6.17 (use Ubuntu 22.04) and **gcc-12** |
| `CSI Probe FAILED: Weka driver not running on host` | install + run the WEKA client on the node (§6a) |
| CSI controller crash-loops on health probes | self-signed API cert → `allowInsecureHttps=true` (lab) or `caCertificate` (prod) |
| `weka local setup container` errors on core allocation | cgroups disabled → pass explicit `--core-ids` |
| No SSD for a new filesystem | shrink the default fs first: `weka fs update default --total-capacity <smaller>` |

---

*Full runnable manifests, the Terraform that builds the lab, per-model verification, and the complete command transcript are in the companion repository:* **`github.com/weka/csi-tenant-onboarding-lab`**